PES UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Campus Lost & Found Management System

**Verified ownership. Explainable matching.
Transactional handover and recoverable document history.**

PROJECT	DETAIL
Course	DBMS Experiential Learning - Level 3
Member 1	Atul Suresh | PES1UG24CS098
Member 2	Aryan Musale | PES1UG24CS085
Technology	Next.js + PostgreSQL + MongoDB
Updated	03 November 2026
Repository	https://github.com/atul-suresh/campus-lost-found-dbms

DBMS Project Report

All sample accounts, reports, ownership statements and handovers use synthetic demonstration data.

01 / Problem, objectives and scope

Lost belongings are often reported through disconnected group chats or informal helpdesk notes. Students may miss relevant reports, and public messages can disclose the very identifying detail needed to verify an owner. A simple list of items does not provide a trustworthy return process.

The proposed system connects lost/found discovery with private ownership evidence, staff review and a recorded handover. It provides a searchable campus directory and ranked suggestions while keeping the final ownership decision with staff.

Objectives

- Centralize lost/found reporting, categorization and location-based discovery.

- Suggest plausible matches using explainable text and metadata ranking.

- Protect identifying clues and claim evidence from unrelated students.

- Prevent duplicate active claims and multiple verified owners for one found item.

- Return an item and close its linked lost report in one SQL transaction.

- Provide audit analytics and retry document updates after temporary database failure.

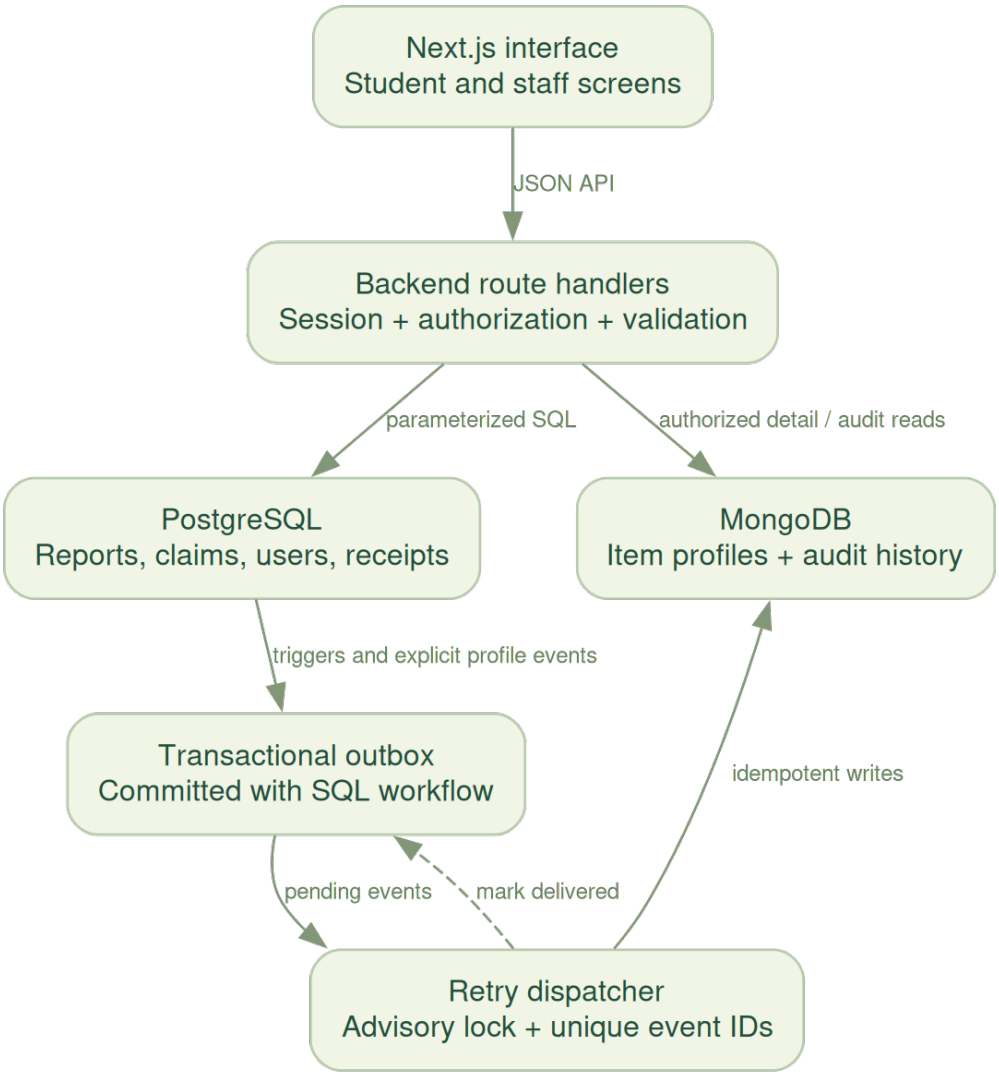
Functional responsibilities

ROLE	CAPABILITIES
Student	Register/sign in; create, edit or delete eligible reports; search; view own matches; submit or withdraw own pending claims; close own lost report.
Staff	View private identifying details and claims; approve/reject evidence; complete in-person handovers; inspect audit aggregation; retry synchronization.

Data and boundaries

The reproducible seed contains 24 synthetic reports, 4 demo accounts, 7 item categories and 6 campus locations. Three sample claims demonstrate pending, approved and handed-over states. No real dataset was supplied. The system does not implement image recognition, email/SMS notifications or automatic ownership verification.

02 / System architecture

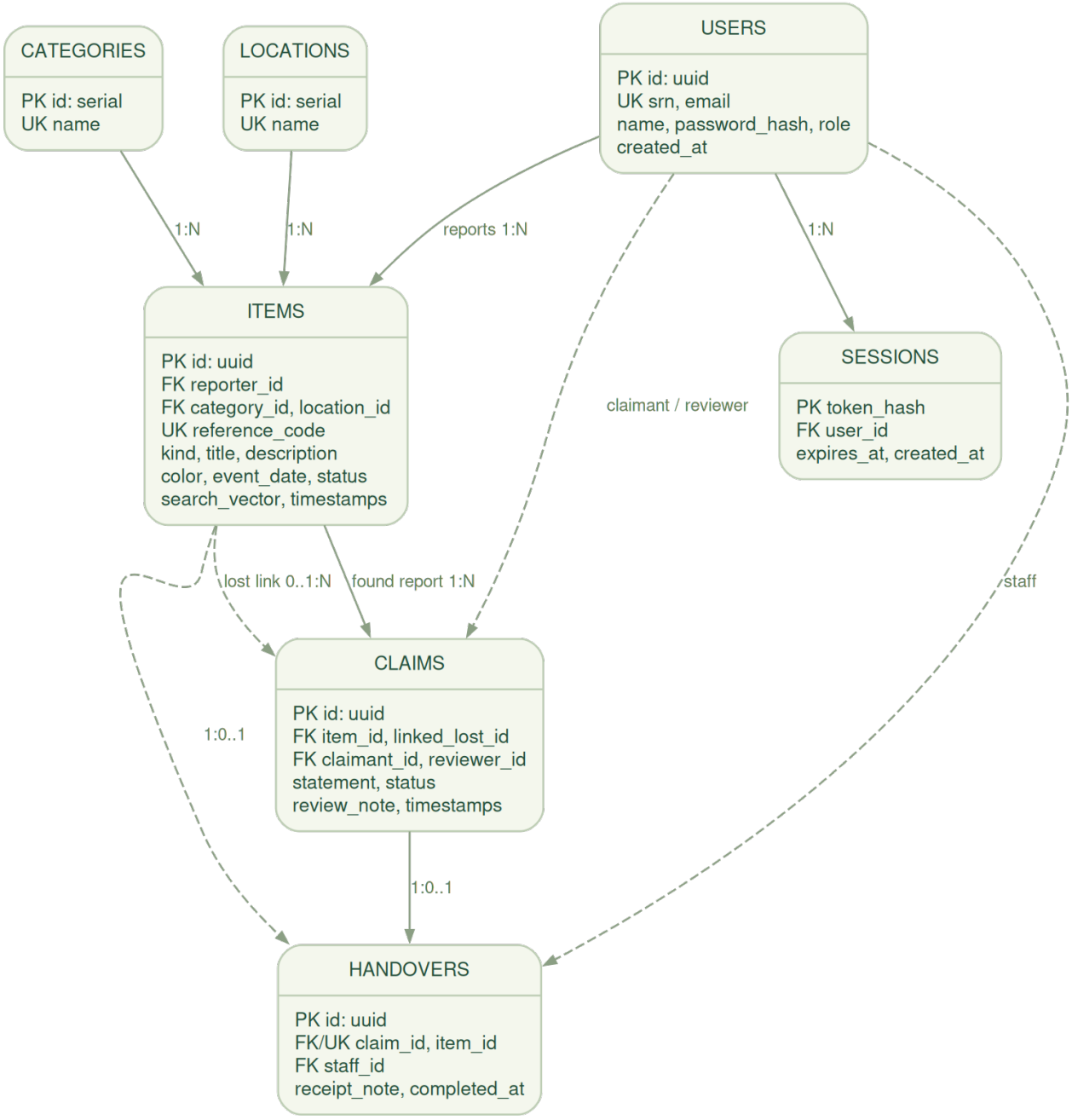


The Next.js page calls backend route handlers. Every protected handler authenticates a server session; services check the role and report ownership. Parameterized PostgreSQL queries operate on normalized workflow records. MongoDB serves item profiles and staff audit documents.

SQL changes and outbox events commit together. A dispatcher writes the event to MongoDB and marks it delivered only after success. Cross-database consistency is eventual; no distributed ACID transaction is claimed. Supplemental details can be pending while the report itself is safely committed.

Configuration uses environment variables. Native PostgreSQL 17.6 and MongoDB 8.0.15 executed the final verification. The delivered laptop configuration uses standard Docker images.

03 / Entity-relationship design



PK = primary key; FK = foreign key; UK = unique key. Each user can report many items and make many claims. Claims reference a found report and optionally a claimant-owned lost report. A verified claim and found item each have at most one handover receipt. Reviewer and handover staff reference the same user table.

The operational diagram shows seven related tables. Two additional infrastructure tables are login_attempts and outbox_events. The outbox entity_id is a polymorphic logical identifier, not an SQL foreign key. Exact column types and constraints are defined in db/schema.sql.

04 / Relational schema and normalization

```
USERS(id PK, name, srn UK, email UK, password_hash, role, created_at)
SESSIONS(token_hash PK, user_id FK, expires_at, created_at)
LOGIN_ATTEMPTS(identifier PK, failures, blocked_until)
CATEGORIES(id PK, name UK)
LOCATIONS(id PK, name UK)
ITEMS(id PK, reporter_id FK, kind, title, description, category_id FK,
      location_id FK, event_date, color, status, created_at, updated_at,
      search_vector GENERATED, reference_code UK)
CLAIMS(id PK, item_id FK, claimant_id FK, linked_lost_id FK, statement,
       status, reviewer_id FK, review_note, created_at, reviewed_at)
HANDOVERS(id PK, claim_id FK/UK, item_id FK/UK, staff_id FK,
          receipt_note, completed_at)
OUTBOX_EVENTS(id PK, event_type, entity_id, payload, created_at,
              processed_at)
```

Normalization

Domain attributes are atomic, and each table uses a single-column primary key, so non-key attributes have no partial dependency on a composite primary key. Category and location names are stored once in lookup tables rather than repeated in reports. User identity is stored independently of reports and claims. These relations follow a practical 3NF design: report facts depend on an item, claim facts on a claim and receipt facts on a handover.

The generated search_vector and reference_code are deliberate derived values for retrieval and display. They are not independent sources of truth. MongoDB profiles intentionally keep a document shape for varying item descriptors. The transient outbox payload is integration infrastructure, not a second normalized item-profile table.

Important integrity rules

RULE	DATABASE ENFORCEMENT
Valid report facts	Non-null columns, bounded text, kind/status checks, category/location/user foreign keys, nonfuture event_date.
One owner	Partial unique index on item_id for approved/handed_over claims.
Duplicate active claim	Partial unique index on item_id and claimant_id for active statuses.
Correct lost-report link	Trigger checks the linked record is open, lost and belongs to the claimant.
Consistent final state	Deferred triggers require approved/reserved and completed-handover/returned state agreement.
Verified handover	Stored function and guard trigger require staff, the correct approved claim and reserved item; receipt/item keys are unique.

05 / Data dictionary: identity and lookups

users

COLUMN	TYPE	NULL	DESCRIPTION
id	uuid	No	Surrogate primary key; UUID for domain records, serial/bigserial for lookup/event records.
name	varchar	No	Display or lookup name; lookup names are unique.
srn	varchar	No	Unique institutional identifier; permitted uppercase characters enforced.
email	varchar	No	Unique lowercase email with database format check.
password_hash	text	No	Salted scrypt hash; no plaintext password stored.
role	varchar	No	student or staff; registration always inserts student.
created_at	timestamptz	No	Creation timestamp with timezone; default now().

sessions

COLUMN	TYPE	NULL	DESCRIPTION
token_hash	character	No	Primary key containing SHA-256 of a random session token.
user_id	uuid	No	Foreign key to users; session removed when user is removed.
expires_at	timestamptz	No	Backend-enforced session expiry timestamp.
created_at	timestamptz	No	Creation timestamp with timezone; default now().

login_attempts

COLUMN	TYPE	NULL	DESCRIPTION
identifier	character	No	Primary key hash of normalized login email.
failures	integer	No	Failed-login counter for temporary throttling.
blocked_until	timestamptz	Yes	Nullable temporary login-block expiry.

Category and location lookups

TABLE	COLUMNS	CONSTRAINTS
categories	id: serial; name: varchar(50)	id primary key; name unique and not null.
locations	id: serial; name: varchar(70)	id primary key; name unique and not null.

06 / Data dictionary: item reports

items

COLUMN	TYPE	NULL	DESCRIPTION
id	uuid	No	Surrogate primary key; UUID for domain records, serial/bigserial for lookup/event records.
reporter_id	uuid	No	Foreign key to users; immutable report owner.
kind	varchar	No	lost or found; cannot change after creation.
title	varchar	No	Public title; trimmed length 3-100.
description	varchar	No	Public description; minimum trimmed length 10; max 1500.
category_id	integer	No	Foreign key to categories.
location_id	integer	No	Foreign key to locations.
event_date	date	No	Date lost/found; must not exceed current_date.
color	varchar	No	Optional public color string; default empty.
status	varchar	No	Workflow state restricted by CHECK and integrity triggers.
created_at	timestamptz	No	Creation timestamp with timezone; default now().
updated_at	timestamptz	No	Maintained by the prepare_item trigger.
search_vector	tsvector	Yes	Generated English tsvector; GIN indexed.
reference_code	text	No	Unique display reference generated at insert.

Items contain public, relational facts. Category-dependent attributes, photo data and private identifying details are stored in MongoDB item_profiles and never appear in the public SQL directory view.

07 / Data dictionary: claims and handovers

claims

COLUMN	TYPE	NULL	DESCRIPTION
id	uuid	No	Surrogate primary key; UUID for domain records, serial/bigserial for lookup/event records.
item_id	uuid	No	Foreign key to the found report; unique in handovers.
claimant_id	uuid	No	Foreign key to users; claimant cannot be the finder.
linked_lost_id	uuid	Yes	Nullable foreign key to claimant-owned open lost report.
statement	varchar	No	Private ownership evidence; trimmed length 12-2000.
status	varchar	No	Workflow state restricted by CHECK and integrity triggers.
reviewer_id	uuid	Yes	Nullable staff user foreign key; required for reviewed claims.
review_note	varchar	No	Staff reason; bounded 500 characters.
created_at	timestamptz	No	Creation timestamp with timezone; default now().
reviewed_at	timestamptz	Yes	Nullable review timestamp; status consistency checked.

handovers

COLUMN	TYPE	NULL	DESCRIPTION
id	uuid	No	Surrogate primary key; UUID for domain records, serial/bigserial for lookup/event records.
claim_id	uuid	No	Unique claim foreign key; one handover per claim.
item_id	uuid	No	Foreign key to the found report; unique in handovers.
staff_id	uuid	No	Foreign key to users; handover trigger requires staff.
receipt_note	varchar	No	Receipt description; minimum trimmed length 5; max 500.
completed_at	timestamptz	No	Completion timestamp; default now().

08 / Outbox and document model

outbox_events

COLUMN	TYPE	NULL	DESCRIPTION
id	bigint	No	Surrogate primary key; UUID for domain records, serial/bigserial for lookup/event records.
event_type	varchar	No	Entity.operation or profile.upsert integration event.
entity_id	uuid	No	UUID logical entity reference; no polymorphic FK.
payload	jsonb	Yes	Pending JSONB event payload; cleared after delivery.
created_at	timestamptz	No	Creation timestamp with timezone; default now().
processed_at	timestamptz	Yes	Null while queued; delivery timestamp after successful projection.

MongoDB collections

COLLECTION	MODEL AND RETRIEVAL
item_profiles	_id: SQL item UUID string; public: flexible attribute object and optional photo; secret: private identifying detail; event_id: latest projection version. Read by exact _id; brand index for descriptor access.
audit_events	_id: unique SQL event sequence number; event_type, entity_id, created_at, details. Staff-only exact/entity history retrieval and grouped action aggregation. Compound entity/time and action/time indexes.

```
{
  "_id": "<SQL item UUID>",
  "public": {"brand": "Milton", "material": "Steel", "photo": null},
  "secret": "Initials engraved under the base",
  "event_id": 52
}
```

The document above is a schema example, not a screenshot or copied record. Strict JSON Schema requires the profile identity, public object, secret string and numeric event version. The backend additionally limits attribute count/key/value lengths, photo type/size and private text length. Public API reads omit secret unless the caller is the reporter or staff.

Advanced NoSQL aggregation

```
audit_events.aggregate([
  {$match: {event_type: {$ne: "profile.upsert"}}},
  {$group: {_id: "$event_type", count: {$sum: 1},
            latest: {$max: "$created_at"}}},
  {$sort: {count: -1, _id: 1}}
])
```

09 / Workflow, privacy and consistency

OBJECT	ALLOWED WORKFLOW
Found report	open -> reserved after staff approval -> returned after receipt.
Lost report	open -> closed by its reporter/staff or linked verified handover.
Claim	pending -> approved, rejected or withdrawn; approved -> handed_over.
Deletion	Only open reports with no claim references/history may be deleted.

Approval and handover

review_claim locks the found item first, then the requested claim. It verifies the staff role and current state, updates the selected claim and reserves the item. Competing pending claims are rejected inside the same statement transaction. confirm_handover checks the reserved/approved pair, inserts a unique receipt, marks the item returned and claim handed_over, then closes an eligible linked lost report.

Deferred consistency triggers examine final table state at commit. Temporary intermediate states within these functions are allowed; inconsistent final states are rejected. A test directly attempted to mark a report reserved without an approved claim and confirmed the statement failed.

Private evidence

Students see public reports, their own identifying detail and their own claim statements. Staff see the evidence needed to verify ownership. The finder does not receive another student's private claim statement through the claim-list API. Audit payloads contain safe event summaries and exclude proof text and passwords.

Recoverable document delivery

A profile event is inserted in the same SQL transaction as its report. The dispatcher uses an advisory transaction lock to serialize workers and locks pending rows. MongoDB audit writes use the SQL event ID as a unique key, and profile writes reject older event versions. After successful writes the SQL delivery timestamp is set and the profile payload is cleared. Failed batches roll back SQL delivery marks, preserving a retryable queue.

An executed failure-recovery test temporarily enforced an incompatible MongoDB validator. The report still committed in SQL with a queued event. Restoring validation and rerunning dispatch created the profile successfully. This proves retry behavior under a rejected document write; it does not simulate every possible distributed failure.

10 / SQL implementation and important queries

The complete executable scripts are db/schema.sql and db/queries.sql. They include CREATE statements, a genuine idempotent ALTER migration for item reference codes, primary/foreign keys, constraints, indexes, views and trigger/function definitions. A disposable table demonstrates DROP without removing application tables.

JOIN with filtering and ordering

```
SELECT reference_code, title, category, location, status
FROM item_directory
WHERE kind='found' AND status='open'
ORDER BY event_date DESC;
```

GROUP BY, HAVING and aggregate

```
SELECT l.name, count(*) AS open_reports
FROM items i JOIN locations l ON i.location_id=l.id
WHERE i.status='open'
GROUP BY l.name HAVING count(*) >= 2
ORDER BY open_reports DESC;
```

Subquery for unclaimed open reports

```
SELECT reference_code, title FROM items i
WHERE kind='found' AND status='open'
AND NOT EXISTS (
  SELECT 1 FROM claims c
  WHERE c.item_id=i.id AND c.status='pending'
);
```

Full-text retrieval

```
SELECT reference_code, title,
  ts_rank(search_vector,
          websearch_to_tsquery('english','blue bottle')) AS rank
FROM items
WHERE search_vector @@ websearch_to_tsquery('english','blue bottle')
ORDER BY rank DESC;
```

INSERT, UPDATE and DELETE are exercised by report creation/edit/deletion and the disposable DDL demonstration. Query result files record executed commands and rows. No query-performance speedup is claimed for the small synthetic dataset.

11 / Ranking and advanced database features

Explainable matching

match_candidates accepts an open lost report and retrieves open found reports from another reporter in the same category. It scores category, location, nonempty color equality, event-date distance and PostgreSQL text rank. The function returns at most 12 candidates scoring at least 35 points.

SIGNAL	POINTS	CALCULATION
Category	35	35 if category IDs are equal.
Location	20	20 if location IDs are equal.
Color	15	15 for equal nonempty colors, case-insensitive.
Date proximity	15	max(0, 15 - min(abs(date difference),15)).
Description/title text	15	min(15, ts_rank_cd(found search_vector, lost text query) * 100).

The score ranges from 0 to 100. A category eligibility check excludes unrelated item types. Color comparison ignores case and surrounding whitespace, and empty colors receive no bonus. The score is heuristic rather than a calibrated ownership probability. Staff must still verify ownership evidence.

Course advanced-feature coverage

FEATURE	CONCRETE USE
Transactions / locks	Report + profile-event write; row-locked approval; atomic receipt/return/link closure.
Views	item_directory JOIN view and category_summary filtered aggregates.
Stored functions	review_claim, confirm_handover and match_candidates.
Triggers	prepare_item, guard_claim, audit emission, handover guard and deferred state checks.
Indexing / full text	GIN search, composite filters, unique reference and partial claim ownership indexes.
NoSQL aggregation	Staff grouped audit action counts and latest activity timestamps.

12 / Backend integration and authentication

The Next.js catch-all route handler exposes a JSON API and delegates database operations to service functions. Credentials are read from environment variables. SQL parameters are bound rather than interpolated. Inputs are checked in the backend and the databases; hiding a button is never treated as authorization.

METHOD / PATH	AUTHORIZED BEHAVIOR
POST auth/register	Create a student only, regardless of submitted role.
POST auth/login; POST auth/logout	Verify password; set session cookie; revoke session.
GET auth/me; GET lookups	Read authenticated user and category/location options.
GET/POST items	Authenticated directory; create a report owned by caller.
GET/PUT/DELETE items/:id	Role-aware detail; reporter/staff edit/delete eligible report.
GET items/:id/matches	Reporter/staff access to open lost-report candidates.
POST items/:id/close	Reporter/staff closes eligible lost report.
GET/POST claims	Own claims for students, all for staff; submit private evidence.
POST claims/:id/withdraw	Claimant-only pending withdrawal.
POST claims/:id/review; handover	Staff-only database verification and receipt workflows.
GET dashboard; GET audit	SQL overview; staff-only audit documents/aggregation.
POST sync; GET diagnostics	Staff-only delivery retry and database diagnostics.

Passwords use salted scrypt hashes. A random session token is stored in an HttpOnly, SameSite cookie; SQL stores its SHA-256 hash and a 12-hour expiry. Login attempts are temporarily blocked after repeated failures. Mutation requests require an allowed Origin. For HTTPS hosting, COOKIE_SECURE must be true. Local compose services bind to loopback and the provided settings are for a laptop demonstration.

13 / Application interface and screenshot evidence

The interface provides student and staff navigation, sign-in/registration, overview counts, a filtered noticeboard, photo/report forms, ranked matches, private claims, staff verification and audit history. Category-specific public descriptors include bottle capacity, device model and clothing size. Editing preserves flexible attributes. The responsive layout uses labelled input controls and clears prior session state on logout.

8 of 8 required browser screenshots are included. The browser suite records 17 passed checks across the student and staff workflow, including desktop and mobile views.

SCREENSHOT FILE	EVIDENCE STATUS
01-login.png	Included: Sign-in page with password hidden
02-student-dashboard.png	Included: SQL-backed student overview
03-search.png	Included: Filtered blue bottle search
04-matches.png	Included: Ranked candidates for open lost report
05-private-claim.png	Included: Synthetic claimant evidence
06-staff-verification.png	Included: Staff verification of identifying detail
07-handover.png	Included: Returned/handed_over states
08-audit.png	Included: Document audit history and grouped counts

The appendix contains the required application screenshots. Original PNGs, including the mobile view, are in evidence/screenshots/.

14 / Actual tests and results

Recorded database, HTTP and browser execution date: 2026-10-29. The raw evidence preserves the original test results.

CHECK	RECORDED RESULT
Next.js production build	See evidence/build.txt for the recorded build output.
Database/service suite	35 passed; 0 failed; 0 skipped, out of 35 tests.
Backend HTTP suite	20 of 20 checks passed against production Next.js server.
Browser workflow suite	17 of 17 checks passed with genuine screenshot capture.
Formatting / attribute checks	3 passed
Dependency audit	0 vulnerabilities reported by the recorded npm audit.
PostgreSQL engine	PostgreSQL 17.6 on x86_64-pc-linux-gnu
MongoDB process	8.0.15
Outbox after checks	0 pending events.
Remaining verification	Native concurrency, browser workflow and required screenshot evidence recorded.

Database cases exercised

Executed cases covered setup idempotence, valid/invalid authentication, expiry/throttling, insert/update/delete, private-field visibility, search/filtering, bad dates, foreign keys, matching, own-report restrictions, duplicate active claims, correct lost links, claimant-only evidence, staff authorization, competing-claim rejection, handover and link closure, premature/duplicate handover, withdrawal, retained claim history, rollback, deferred consistency, MongoDB validation, idempotent event replay, delivery failure/recovery, aggregation/views and DDL/query examples.

Native PostgreSQL simultaneous-approval test passed in the recorded run.

Recorded HTTP cases

The 20 production HTTP checks exercised anonymous rejection, Origin enforcement, protected session cookies, dashboard/directory retrieval, staff-only routes, invalid input, parameterized search, student-only registration, logout, audit/diagnostics and synchronization. The full lifecycle created a found report and linked lost report, hid the private clue from a student, submitted ownership evidence, denied student review, then approved and handed over as staff. Both final report states were checked.

Raw outputs: evidence/test_results.tap, http_results.json, query_results.jsonl, build.txt and verification.json. The verification database includes additional isolated test records beyond the 24-report seed. Test data counts are not presented as real campus activity.

15 / Setup, demonstration and team work

Local execution

Install Node.js 20.9 or newer and start Docker Desktop. Open the extracted project folder, then launch START_PROJECT.command. The launcher prepares the databases and a fresh synthetic demo pair, builds the application and starts the server.

```
bash START_PROJECT.command
```

Open http://localhost:3000. The example demo password is CampusDemo@2026 and may be changed in .env before seeding. Demo emails are atul@campus.test, student085@campus.test, student@campus.test and staff@campus.test. The actual seed generates hashes from DEMO_PASSWORD; no production credential is embedded in application code.

Demonstration sequence

Show an open lost report and matching found candidates. Submit a new claim with a private identifying detail and a linked lost report. Sign in as staff, compare evidence, approve with a reason, and complete handover with a receipt. Show returned/handed_over and closed linked-report states. Explain SQL constraints/functions, display the audit aggregation, and run npm run demo:queries.

Component ownership for demonstration

MEMBER	DATABASE + APPLICATION WORK
Atul Suresh (PES1UG24CS098)	SQL schema/constraints, matching/verification; report/search/matches and staff workflow. Explain category eligibility, normalized color ranking and the regression tests.
Aryan Musale (PES1UG24CS085)	MongoDB validation/aggregation/retry; authentication, profile/photo forms and audit view. Explain category-specific descriptors, profile validation and recovery checks.

The final implementation and automated verification were completed with coding assistance. Component ownership above organizes the student demonstration. Individual review, understanding and any personally authored modifications must be recorded by each member in docs/TEAM_CONTRIBUTIONS.md.

16 / Conclusion

The custom implementation connects relational integrity, document flexibility, explainable discovery and verified recovery. It goes beyond report CRUD through private claims, atomic handovers, final-state enforcement and retryable audit delivery. Automated evidence supports the implemented database and API behavior under the stated test environment.

The final implementation supports reporting, discovery, private ownership review and consistent handover. The supplied tests exercise the recovery workflow, integrity rules, authorization and document-delivery recovery using synthetic data.

Verification environment

Verification used native PostgreSQL 17.6 and MongoDB 8.0.15 through loopback TCP in an isolated build container. Standard upstream Docker images are provided for the laptop demonstration. Detailed runtime notes are recorded in evidence/verification.json. Automated tests cover the stated synthetic scenarios rather than a production deployment.

References

Course source: Supplied DBMS Level 3 Project Requirements.pdf, pp. 1-5; supplied instructor email image.

Next.js: https://nextjs.org/docs/app/api-reference/functions/cookies

Next.js authentication: https://nextjs.org/docs/app/guides/authentication

PostgreSQL trigger definitions: https://www.postgresql.org/docs/current/sql-createtrigger.html

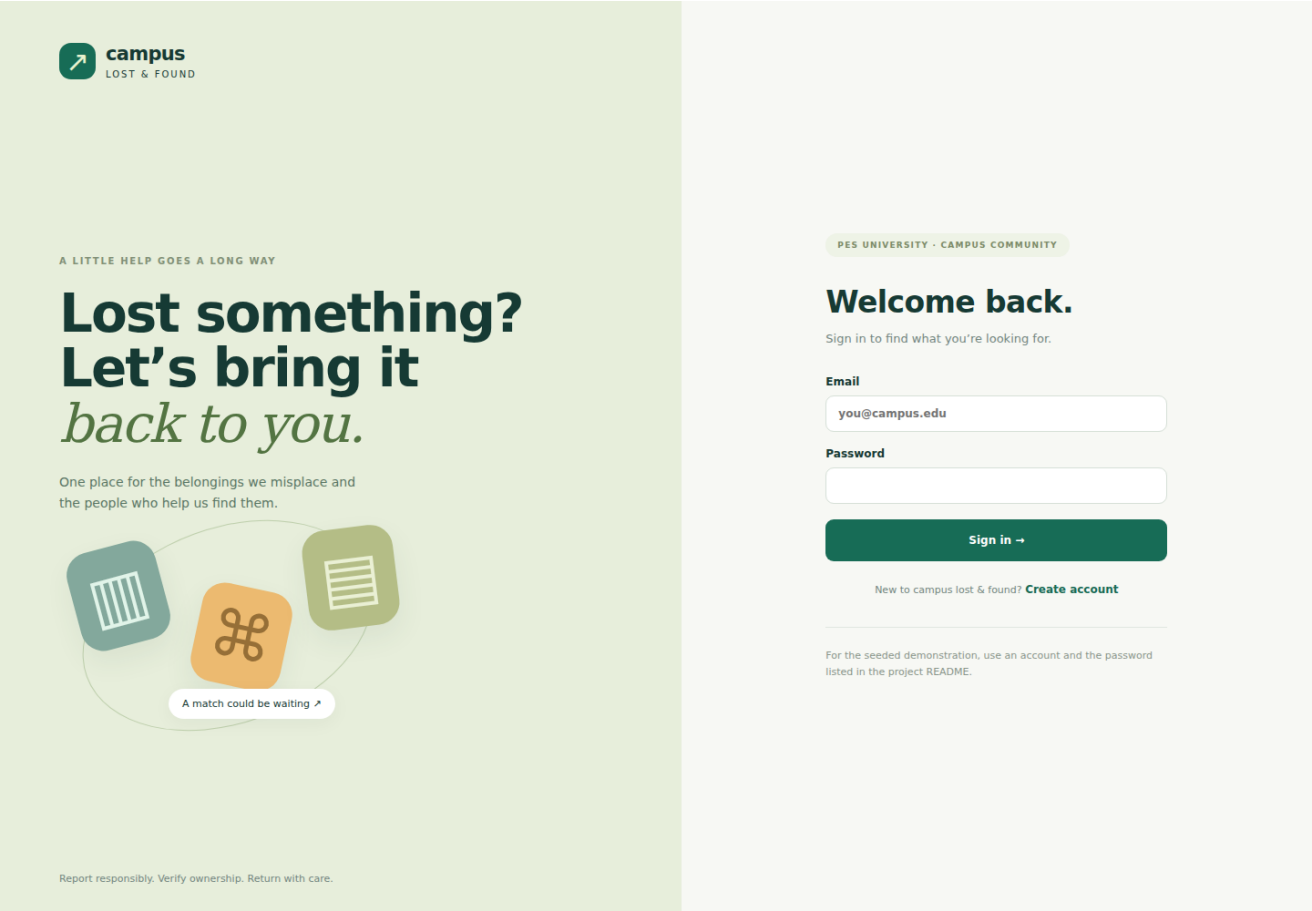
PostgreSQL text search: https://www.postgresql.org/docs/current/textsearch.html

MongoDB validation: https://www.mongodb.com/docs/manual/core/schema-validation/

MongoDB aggregation: https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

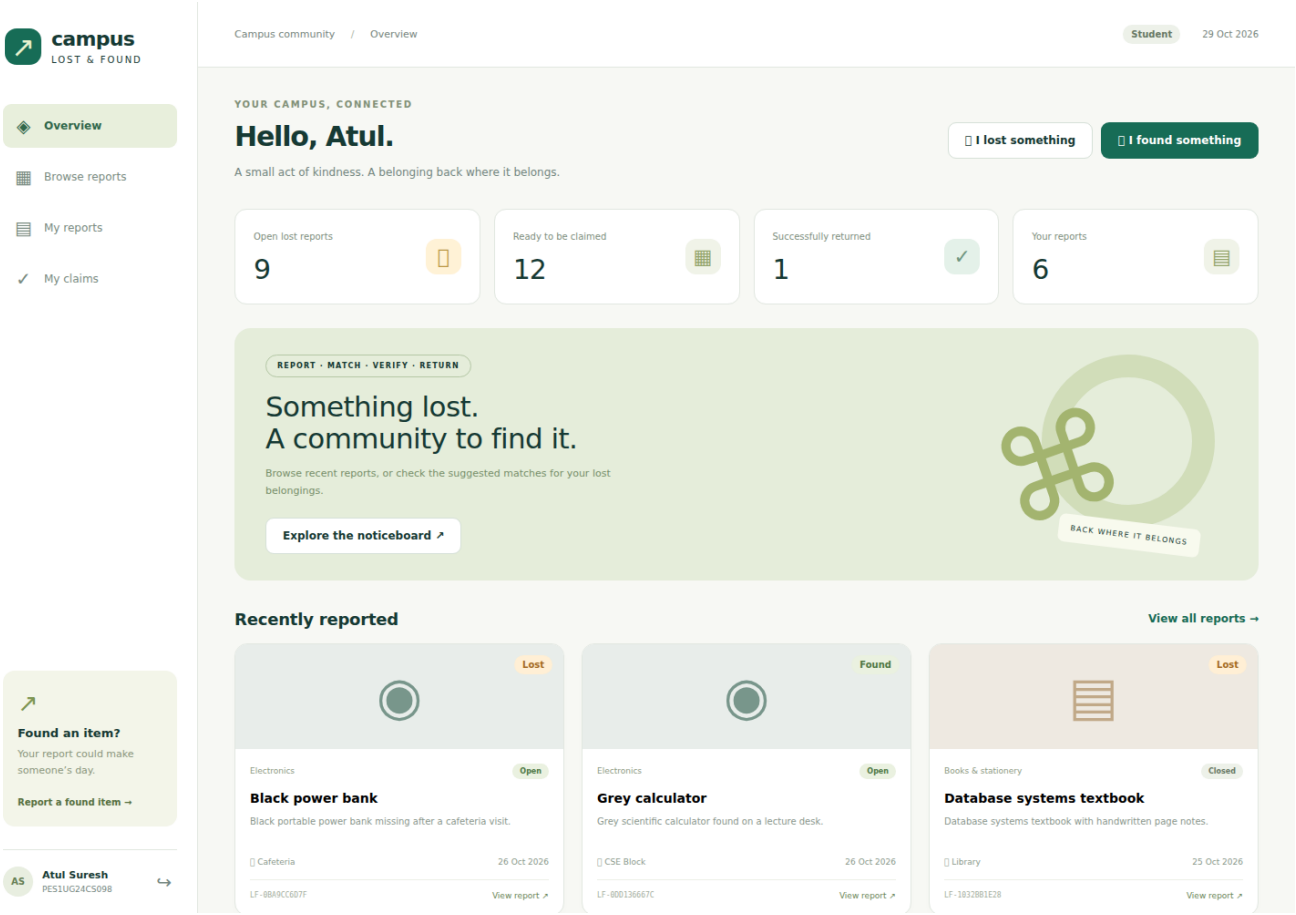
17 / Captured application screenshots

Sign-in page with password hidden



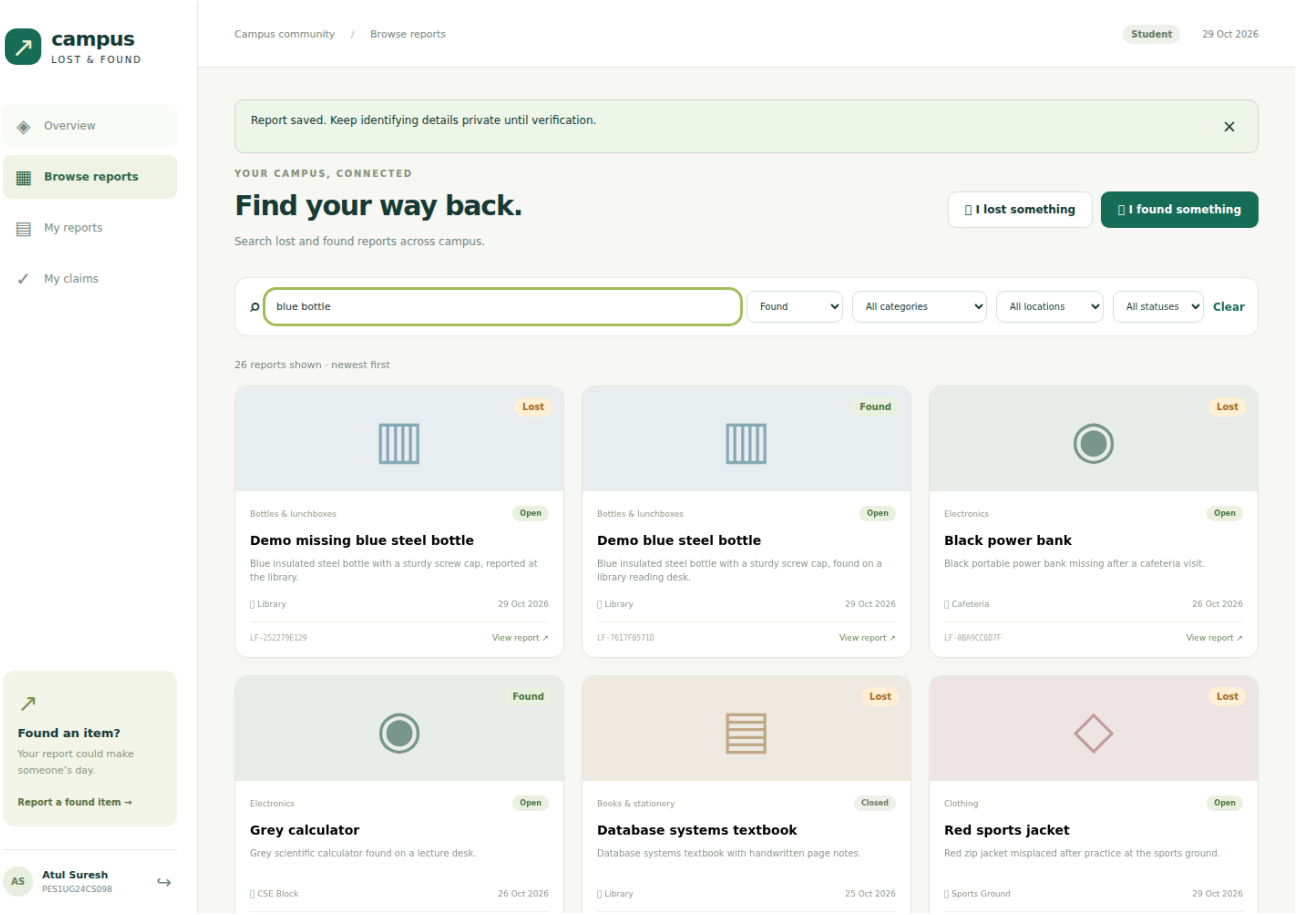
Actual locally captured browser screenshot: 01-login.png

SQL-backed student overview



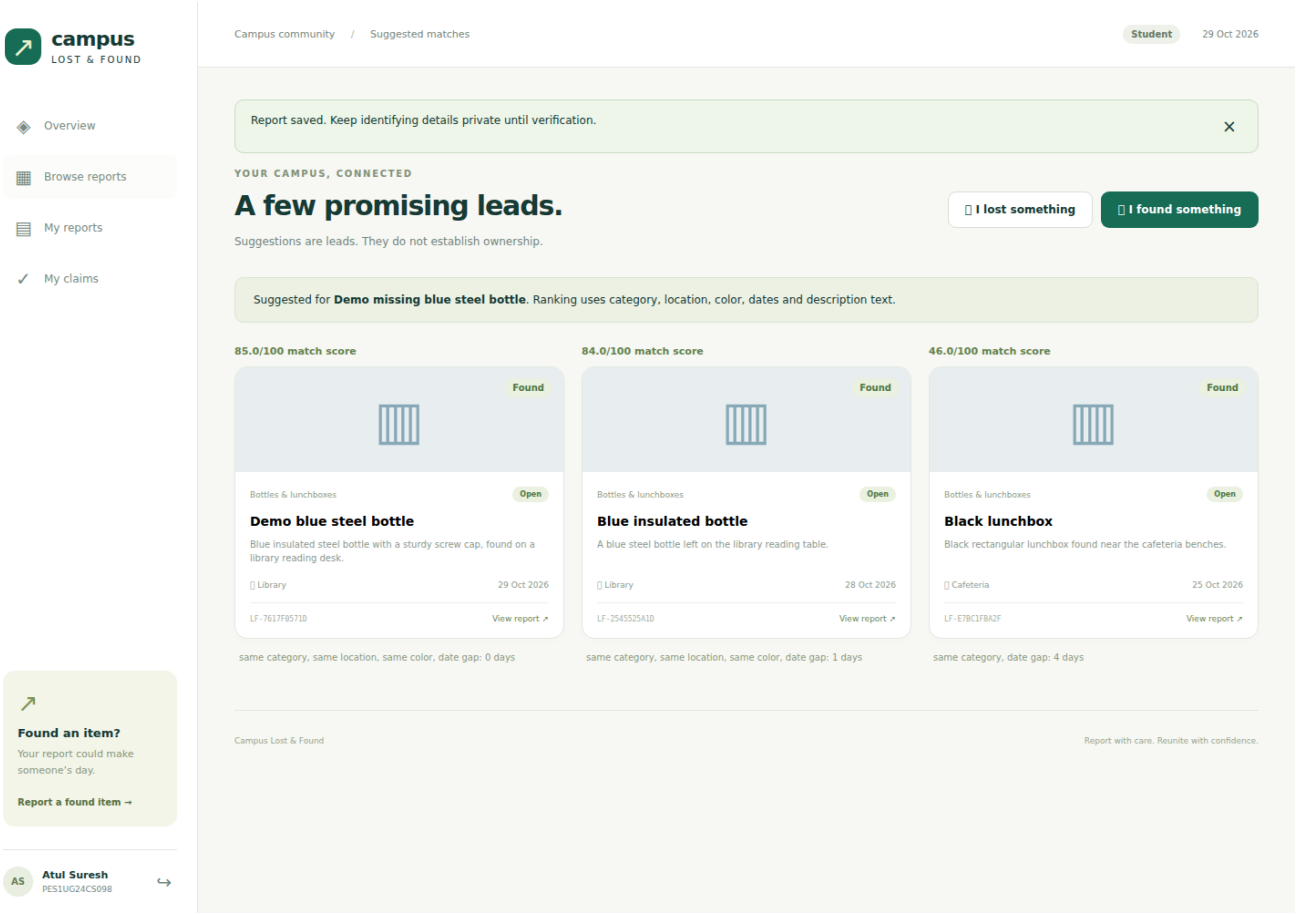
Actual locally captured browser screenshot: 02-student-dashboard.png

Filtered blue bottle search



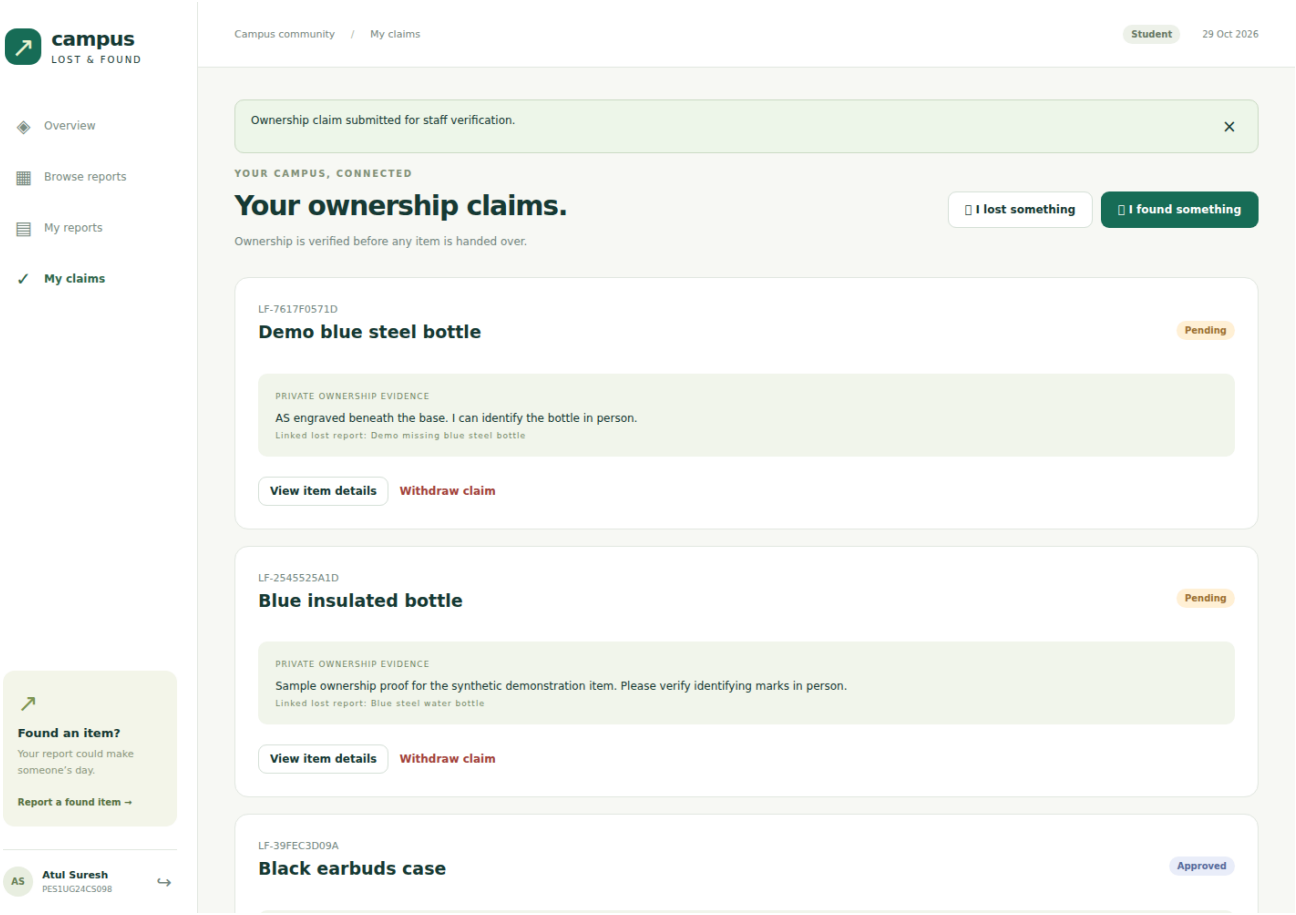
Actual locally captured browser screenshot: 03-search.png

Ranked candidates for open lost report



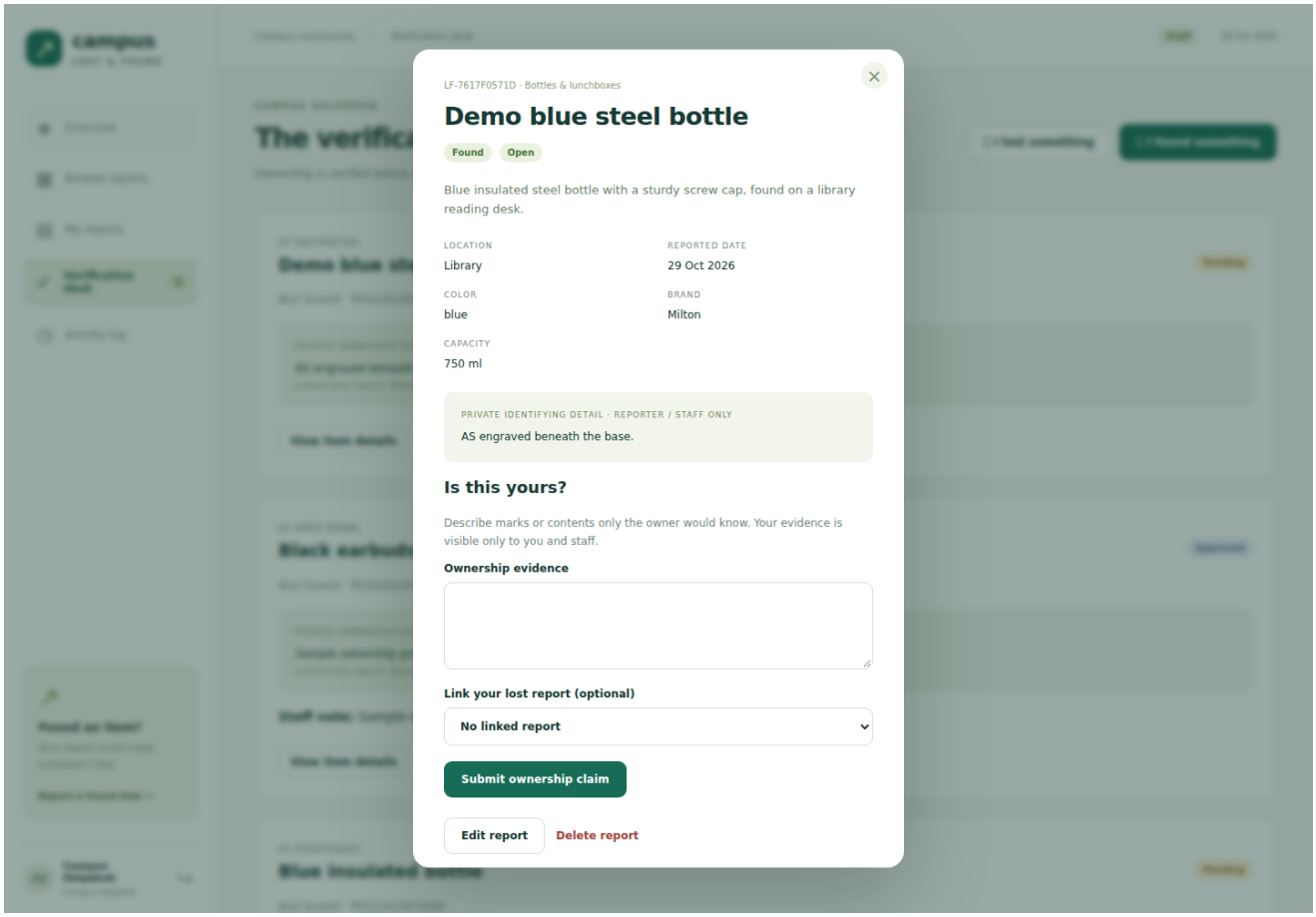
Actual locally captured browser screenshot: 04-matches.png

Synthetic claimant evidence



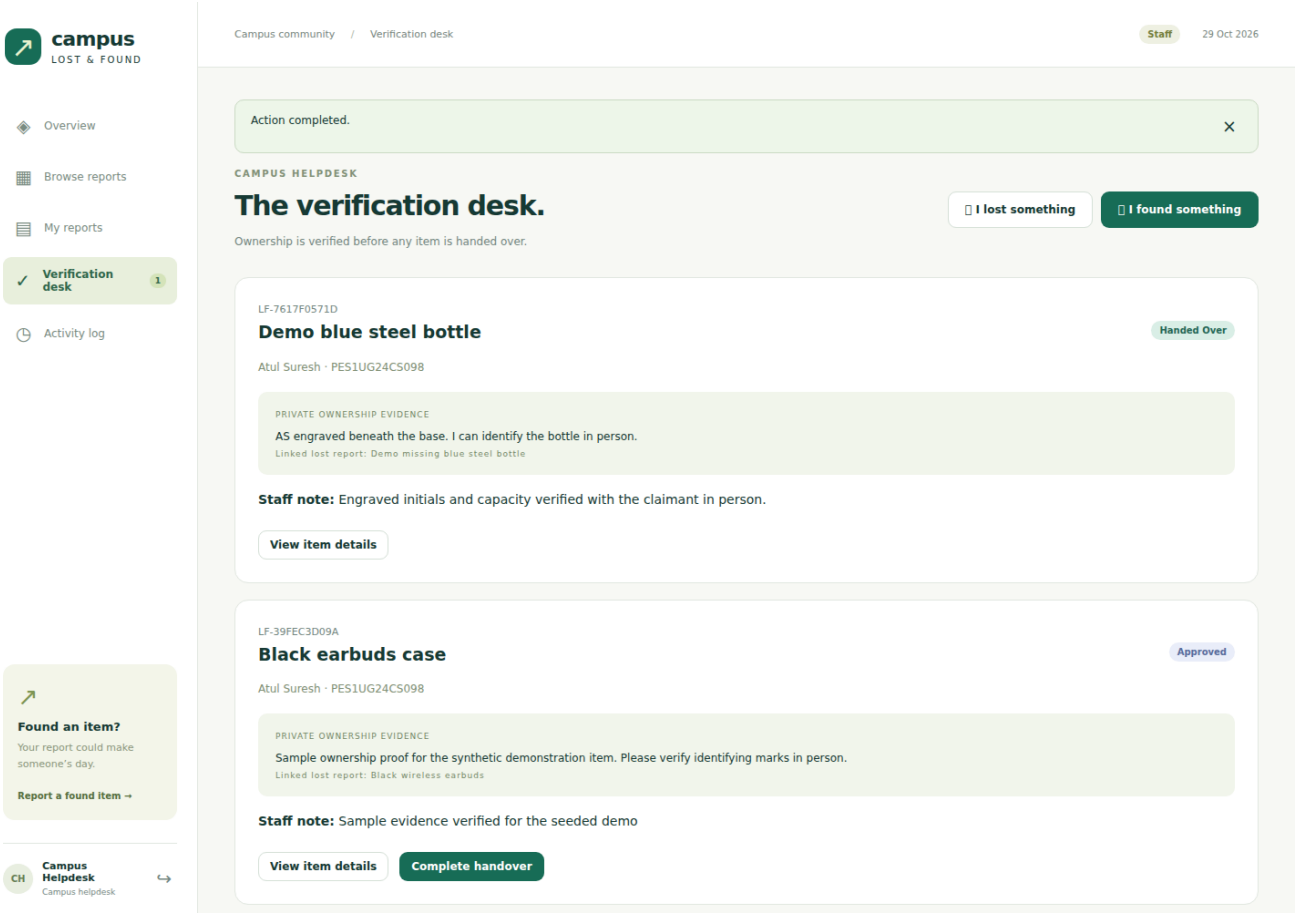
Actual locally captured browser screenshot: 05-private-claim.png

Staff verification of identifying detail



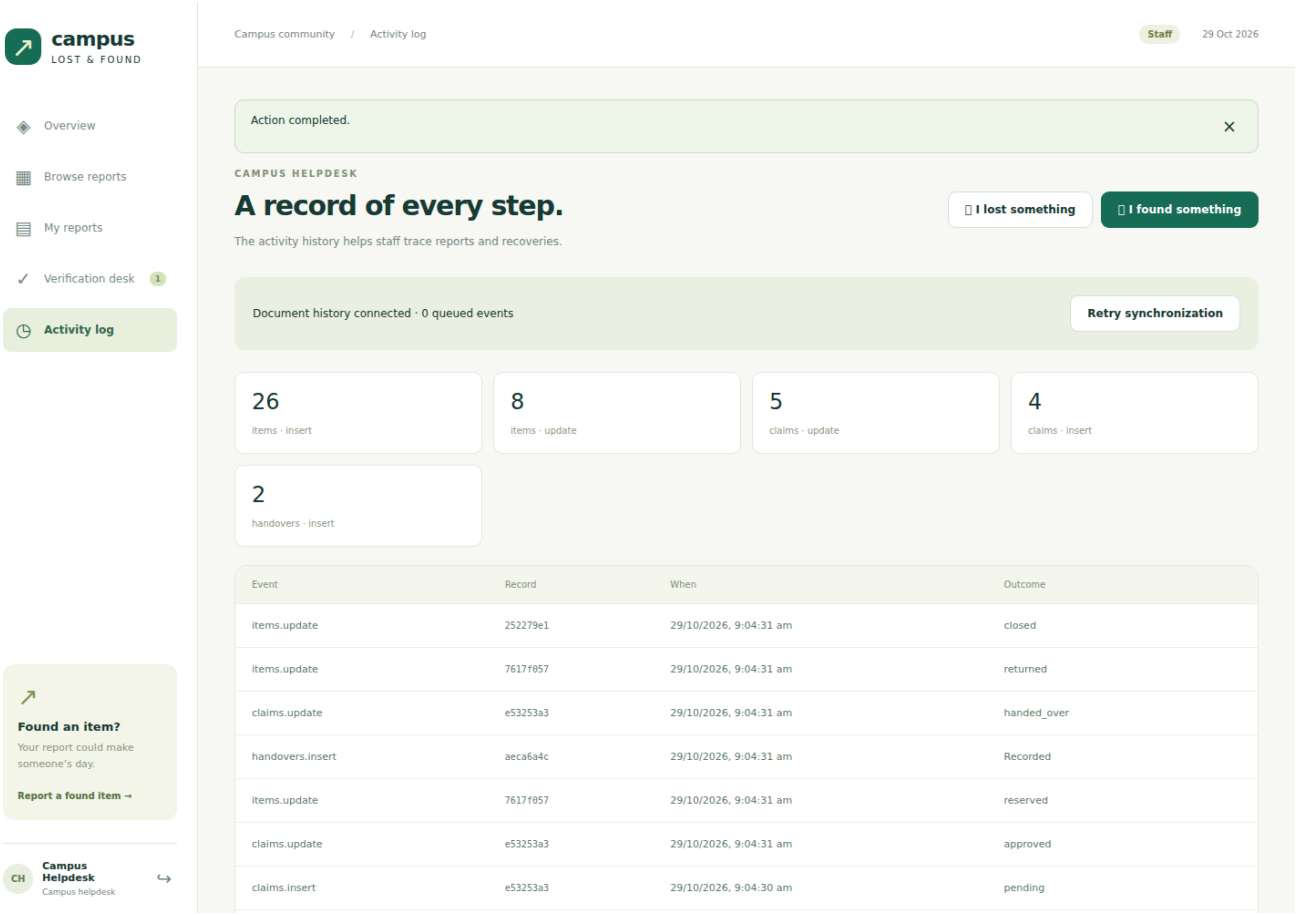
Actual locally captured browser screenshot: 06-staff-verification.png

Returned/handed_over states



Actual locally captured browser screenshot: 07-handover.png

Document audit history and grouped counts



Actual locally captured browser screenshot: 08-audit.png